Machine Learning

Lab Manual

Himanshu Sardana 102303244

Table of Contents

Assignment 1

Question 1 3
Question 2 5
Question 3 6
Question 4 7

Assignment 2

Question 1. (a) 8
Question 1. (a) 8
Question 1. (b) 9
Question 1. (c) 9
Question 2 10
Question 3 11

Assignment 3

Question 1 13
Question 2 15

Assignment 4

Question 1 20
Question 2 21
Question 3 22

Assignment 5

Question 1 24
Question 2 25
Question 3 26

Assignment 6

Question 1 27
Question 2 29
Question 3 30

Assignment 7

Question 1 32
Question 2 38

Assignment 8

Question 1 46
Question 2 47

Assignment 1

Question 1

(a) Reverse the NumPy array: `arr = np.array([1, 2, 3, 6, 4, 5])`

(b) Flatten the NumPy arr: `array1 = np.array([[1, 2, 3], [2, 4, 5], [1, 2, 3]])` using any two NumPy in-built methods

(c) Compare the following numpy arrays: `arr1 = np.array([[1, 2], [3, 4]])` `arr2 = np.array([[1, 2], [3, 4]])`

(d) Find the most frequent value and their indice(s) in the following arrays:

- `x = np.array([1,2,3,4,5,1,2,1,1,1]`
- `y = np.array([1, 1, 1, 2, 3, 4, 2, 4, 3, 3, ])`

(e) For the array `gfg = np.matrix('[4, 1, 9; 12, 3, 1; 4, 5, 6]')`, find

- Sum of all elements
- Sum of all elements, row-wise
- Sum of all elements, column-wise

(f) For the matrix: `n_array = np.array([[55, 25, 15],[30, 44, 2],[11, 45, 77]])`, find

- Sum of diagonal elements
- Eigen values of matrix
- Eigen vectors of matrix
- Inverse of matrix
- Determinant of matrix

(g) Multiply the following matrices and also find covariance between matrices using NumPy:

(a)
```
p = [[1, 2], [2, 3]]
q = [[4, 5], [6, 7]]
```

(b)
```
p = [[1, 2], [2, 3], [4, 5]]
q = [[4, 5, 1], [6, 7, 2]
```

(h) For the matrices: `x = np.array([[2, 3, 4], [3, 2, 9]])`; `y = np.array([[1, 5, 0], [5, 10, 3]])`, find inner, outer and cartesian product?

Solution

```python
import numpy as np

# (a)
arr = np.array([1, 2, 3, 6, 4, 5])
arr = arr[::-1]
print(arr)

# (b)
arr = np.array([[1,2,3], [2, 4, 5], [1, 2, 3]])
arr = arr.flatten()
print(arr)

# (c)
arr1 = np.array([[1, 2,], [3, 4]])
arr2 = np.array([[1, 2], [3, 4]])

print(arr1 == arr2)

# (d)
```

```
x = np.array([1, 2, 3, 4, 5, 1, 2, 1, 1, 1])
y = np.array([1, 21, 1, 2, 3, 4, 2, 4, 3, 3])

counts = {}
for i in x:
    counts[i] = counts.get(i, 0) + 1
print(max(counts, key=counts.get))

counts = {}
for i in y:
    counts[i] = counts.get(i, 0) + 1
print(max(counts, key=counts.get))

# (e)
gfg = np.matrix('[4, 1, 9; 12, 3, 1; 4, 5, 6]')
print(np.sum(gfg))
print(np.sum(gfg, axis=0))
print(np.sum(gfg, axis=1))

# (f)
n_array = np.array([[55, 25, 15], [30, 44, 2], [11, 45, 77]])
print(np.trace(n_array))
eigenvalues, eigenvectors = np.linalg.eig(n_array)
print("Eigenvalues:", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
inverse = np.linalg.inv(n_array)
print("Inverse of the matrix:\n", inverse)
det = np.linalg.det(n_array)

# (g)
p = np.array([[1, 2], [2, 3]])
q = np.array([[4, 5], [6, 7]])

product = np.multiply(p, q)
covariance = np.cov(p, q, rowvar=False)

print( product)
print(covariance)

# (h)
x = np.array([[2, 3, 4], [3, 2, 9]])
y = np.array([[1, 5, 0], [5, 10, 3]])

inner_product = np.inner(x, y)
outer_product = np.outer(x, y)

print("Inner Product:\n", inner_product)
print("Outer Product:\n", outer_product)
```

Output

```
[5 4 6 3 2 1]
[1 2 3 2 4 5 1 2 3]
[[ True  True]
 [ True  True]]
1
3
45
[[20  9 16]]
```

```
[[14]
 [16]
 [15]]
176
Eigenvalues: [98.16835147 28.097044   49.73460452]
Eigenvectors:
 [[ 0.4574917   0.34637121 -0.15017693]
 [ 0.28447814 -0.72784061 -0.4852124 ]
 [ 0.84248058  0.59184038  0.8614034 ]]
Inverse of the matrix:
 [[ 0.02404141 -0.00911212 -0.00444671]
 [-0.01667882  0.02966905  0.0024785 ]
 [ 0.00631287 -0.01603732  0.01217379]]
[[ 4 10]
 [12 21]]
[[0.5 0.5 1.  1. ]
 [0.5 0.5 1.  1. ]
 [1.  1.  2.  2. ]
 [1.  1.  2.  2. ]]
Inner Product:
 [[17 52]
 [13 62]]
Outer Product:
 [[ 2 10  0 10 20  6]
 [ 3 15  0 15 30  9]
 [ 4 20  0 20 40 12]
 [ 3 15  0 15 30  9]
 [ 2 10  0 10 20  6]
 [ 9 45  0 45 90 27]]
```

QUESTION 2

For the array: `array = np.array([[1, -2, 3],[-4, 5, -6]])`

(a) Find element-wise absolute value

(b) Find the 25th, 50th, and 75th percentile of flattened array, for each column, for each row.

(c) Mean, Median and Standard Deviation of flattened array, of each column, and each row

(d) For the array: a = np.array([−1.8, −1.6, −0.5, 0.5,1.6, 1.8, 3.0]). Find floor, ceiling and truncated value, rounded values

SOLUTION

```python
import numpy as np

# (a)
arr = np.array([[1, -2, 3], [-4, 5, -6]])

abs_values = np.abs(arr)
print(abs_values)

perc_25_row = np.percentile(arr, 25, axis=1)
perc_25_col = np.percentile(arr, 25, axis=0)
print(perc_25_row)
print(perc_25_col)

perc_50_row = np.percentile(arr, 50, axis=1)
perc_50_col = np.percentile(arr, 50, axis=0)
```

```python
print(perc_50_row)
print(perc_50_col)

perc_75_row = np.percentile(arr, 75, axis=1)
perc_75_col = np.percentile(arr, 75, axis=0)
print(perc_75_row)
print(perc_75_col)

flattened_arr = arr.flatten()
print(np.mean(flattened_arr))
print(np.median(flattened_arr))
print(np.std(flattened_arr))

# (b)
a = np.array([-1.8, -1.6, -0.5, 0.5, 1.6, 1.8, 3.0])
floor_values = np.floor(a)
ceil_values = np.ceil(a)
trunc_values = np.trunc(a)
rounded_values = np.round(a)

print(floor_values)
print(ceil_values)
print(trunc_values)
print(rounded_values)
```

OUTPUT

```
[[1 2 3]
 [4 5 6]]
[-0.5 -5. ]
[-2.75 -0.25 -3.75]
[ 1. -4.]
[-1.5  1.5 -1.5]
[2.  0.5]
[-0.25  3.25  0.75]
-0.5
-0.5
3.8622100754188224
[-2. -2. -1.  0.  1.  1.  3.]
[-1. -1. -0.  1.  2.  2.  3.]
[-1. -1. -0.  0.  1.  1.  3.]
[-2. -2. -0.  0.  2.  2.  3.]
```

QUESTION 3

(a) For the array: `array = np.array([10, 52, 62, 16, 16, 54, 453])`, find
   (a) Sorted array
   (b) Indices of sorted array
   (c) 4 smallest elements
   (d) 5 largest elements

(b) For the array: `array = np.array([1.0, 1.2, 2.2, 2.0, 3.0, 2.0])`, find
   (a) Integer elements only
   (b) Float elements only

Solution

```
import numpy as np

# (a)
arr = np.array([10, 52, 62, 16, 16, 54, 453])
sorted_arr = np.sort(arr)

print(sorted_arr)
print(sorted_arr[:4])
print(sorted_arr[::-1][:5])

# (b)
arr = np.array([1.0, 1.2, 2.2, 2.0, 3.0, 2.0])
print(arr)

arr_int = [i for i in arr if i.is_integer()]
print(arr_int)

arr_float = [i for i in arr if i.is_integer() is False]
print(arr_float)
```

Output

```
[ 10  16  16  52  54  62 453]
[10 16 16 52]
[453  62  54  52  16]
[1.  1.2 2.2 2.  3.  2. ]
[np.float64(1.0), np.float64(2.0), np.float64(3.0), np.float64(2.0)]
[np.float64(1.2), np.float64(2.2)]
```

Question 4

Write a function named img_to_array(path) that reads an image from a specified path and save it as text file on local machine? (Note: use separate cases for RGB and Grey Scale images)

Solution

```
import numpy as np
import cv2

def img_to_array(path):
    img = cv2.imread(path)
    with open(f"{path}.txt", "a+") as f:
        f.write(str(img))

img_to_array("../assignment_01/screenshots.png")
```

Assignment 2

Dataset Link: https://www.kaggle.com/jahias/microsoft-adventure-works-cycles-customer-data

Question 1. (a)

Based on Feature Selection, Cleaning, and Preprocessing to Construct an Input from Data Source

(a) Examine the values of each attribute and Select a set of attributes only that would affect to predict future bike buyers to create your input for data mining algorithms. Remove all the unnecessary attributes. (Select features just by analysis).

Solution

```
import pandas as pd
import numpy as np

df = pd.read_csv('./AWCustomers.csv')
print(df.columns)
```

Output

```
Index(['CustomerID', 'Title', 'FirstName', 'MiddleName', 'LastName', 'Suffix',
'AddressLine1', 'AddressLine2', 'City', 'StateProvinceName',
'CountryRegionName', 'PostalCode', 'PhoneNumber', 'BirthDate',
'Education', 'Occupation', 'Gender', 'MaritalStatus', 'HomeOwnerFlag',
'NumberCarsOwned', 'NumberChildrenAtHome', 'TotalChildren',
'YearlyIncome', 'LastUpdated'],
```

Question 1. (a)

Select relevant Attributes

Solution

(a) **Age**: Age strongly influences lifestyle, mobility needs, and health goals.
(b) **Education**: Education level often correlates with awareness of health, environmental issues, and sustainable transport benefits.
(c) **Occupation**: Work type affects commute distance, income, and time availability.
(d) **Gender**: Gender can reflect differences in purchasing motivations, preferred cycle types, and
(e) **MaritalStatus**: Family structure affects spending priorities and intended use.
(f) **HomeOwnerFlag**: Homeownership can be a proxy for financial stability and storage availability.
(g) **NumberCarsOwned**: Indicates transport preferences and potential openness to cycling.
(h) **YearlyIncome**: ncome determines affordability and the likelihood of purchasing premium or multiple bicycles.

∴ selecting only these attributes for the input DataFrame.

Question 1. (b)

Based on Feature Selection, Cleaning, and Preprocessing to Construct an Input from Data Source

(b) Create a new Data Frame with the selected attributes only.

Solution

```
selected_features = [
    'age',
    'education',
    'occupation',
    'gender',
    'maritalstatus',
    'homeownerflag',
    'numbercarsowned',
    'yearlyincome'
]

df_selected = df[selected_features].copy()
```

Question 1. (c)

Based on Feature Selection, Cleaning, and Preprocessing to Construct an Input from Data Source

(c) Determine a Data value type (Discrete, or Continuous, then Nominal, Ordinal, Interval, Ratio) of each attribute in your selection to identify preprocessing tasks to create input for your data mining.

Solution

1. **Age**: Continuous (Ratio) because it is measured in years with a true zero and equal intervals, allowing meaningful comparisons. Age reflects lifestyle stage, fitness levels, and transport needs, influencing whether cycling is for commuting, sport, or leisure.
2. **Education**: Discrete (Ordinal) because it consists of ordered categories (e.g., high school, bachelor's, master's) with no fixed interval between them. Education level often correlates with health awareness and environmental consciousness, affecting likelihood of bicycle purchases.
3. **Occupation**: Discrete (Nominal) because it categorizes individuals without intrinsic order (e.g., engineer, teacher, manager). Work type impacts commute habits, income level, and time available for cycling.
4. **Gender**: Discrete (Nominal) because it is a categorical variable with no inherent order. Gender may influence bicycle preferences, design choices, and responsiveness to marketing.
5. **MaritalStatus**: Discrete (Nominal) because it categorizes without ranking (e.g., single, married, divorced). Household structure affects spending priorities and whether purchases are for individuals or families.
6. **HomeOwnerFlag**: Discrete (Binary) because it has only two possible values (yes/no). Homeownership can indicate financial stability and the availability of space for bicycle storage.

7. **NumberCarsOwned**: Discrete (Ratio) because it has a true zero and allows ratio comparisons. It serves as a proxy for transportation reliance—fewer cars may indicate cycles are used for commuting, more cars for recreation.
8. **YearlyIncome**: Continuous (Ratio) because it has a true zero and equal intervals, allowing meaningful ratios. Income directly affects affordability and the type of bicycle purchased.

Question 2

Depending on the data type of each attribute, transform each object from your preprocessed data. Use all the data rows (= 18000 rows) with the selected features as input to apply all the tasks below, do not perform each task on the smaller data set that you got from your random sampling result.

1. Handling Null values
2. Normalization
3. Discretization (Binning) on Continuous attributes or Categorical Attributes with too many different values
4. Standardization/Normalization
5. Binarization (One Hot Encoding)

Solution

(a) Handling Null Values
Checking for Null values in the DataFrame:

```
print(df_selected.isnull().sum())
```

We have no null values in the selected features, so no action is needed.

However, if there were null values, we could handle them by either dropping the rows or filling them with appropriate values (imputation). This can be done as follows:
(a) Dropping rows with null values:

```
df_selected.dropna(inplace=True)
```

(b) Filling null values with the mean (for continuous variables) or mode (for categorical variables):

```
for col in df_selected.columns:
    if df_selected[col].dtype == 'object':  # Categorical
        df_selected[col].fillna(df_selected[col].mode()[0], inplace=True)
    else:  # Continuous
        df_selected[col].fillna(df_selected[col].mean(), inplace=True)
```

(c) Normalization

```
from sklearn.preprocessing import MinMaxScaler

numeric_cols = df_selected.select_dtypes(include=[np.number]).columns
scaler = MinMaxScaler()
df_selected[numeric_cols] = scaler.fit_transform(df_selected[numeric_cols])
print(df_selected.head())
```

(c) Discretization (Binning) on Continuous attributes or Categorical Attributes with too many different values

```
df_selected['Age_binned'] = pd.cut(df_selected['Age'], bins=4, labels=False)
```

(d) Standardization/Normalization

```
std_scaler = StandardScaler()
df_selected[numeric_cols] = std_scaler.fit_transform(df_selected[numeric_cols])
```

(e) Binarization (One Hot Encoding)

```
categorical_cols = df_selected.select_dtypes(exclude=[np.number]).columns
encoder = OneHotEncoder(sparse_output=False)
encoded_data = encoder.fit_transform(df_selected[categorical_cols])
encoded_df = pd.DataFrame(encoded_data,
columns=encoder.get_feature_names_out(categorical_cols))
df_final = pd.concat([df_selected.drop(columns=categorical_cols), encoded_df], axis=1)

print(df_final.columns)
```

OUTPUT

```
Age                0
Education          0
Occupation         0
Gender             0
MaritalStatus      0
HomeOwnerFlag      0
NumberCarsOwned    0
YearlyIncome       0
dtype: int64
```

```
Age  HomeOwnerFlag  NumberCarsOwned  YearlyIncome
0  0.185714            1.0              0.6      0.496842
1  0.400000            1.0              0.4      0.489453
2  0.214286            0.0              0.6      0.536172
3  0.328571            1.0              0.4      0.317083
4  0.357143            1.0              0.2      0.231958
```

```
Age_binned
0    9555
1    7208
2    1544
3      54
Name: count, dtype: int64
```

```
Age  HomeOwnerFlag  NumberCarsOwned  YearlyIncome
0 -0.482516       0.798603         1.892524      0.298555
1  0.851033       0.798603         0.798389      0.271180
2 -0.304710      -1.252187         1.892524      0.444261
3  0.406517       0.798603         0.798389     -0.367401
4  0.584324       0.798603        -0.295746     -0.682765
```

```
Index(['Age', 'HomeOwnerFlag', 'NumberCarsOwned', 'YearlyIncome', 'Age_binned',
'Education_Bachelors', 'Education_Graduate Degree',
'Education_High School', 'Education_Partial College',
'Education_Partial High School', 'Occupation_Clerical',
'Occupation_Management', 'Occupation_Manual', 'Occupation_Professional',
'Occupation_Skilled Manual', 'Gender_F', 'Gender_M', 'MaritalStatus_M',
'MaritalStatus_S'],
dtype='object')
```

QUESTION 3

Make sure each attribute is transformed in a same scale for numeric attributes and Binarization for each nominal attribute, and each discretized numeric attribute to

standardization. Make sure to apply a correct similarity measure for nominal (one hot encoding)/binary attributes and numeric attributes respectively.

1. Calculate Similarity in Simple Matching, Jaccard Similarity, and Cosine Similarity between two following objects of your transformed input data.
2. Calculate Correlation between two features Commute Distance and Yearly Income

Solution

1. Calculating Similarity Measures between the first two columns of the transformed DataFrame:

```
obj1 = df_final.iloc[0].values.reshape(1, -1)
obj2 = df_final.iloc[1].values.reshape(1, -1)

smc = (obj1 == obj2).sum() / len(obj1[0])
cos_sim = cosine_similarity(obj1, obj2)[0][0]
jac_sim = 1 - jaccard(encoded_df.iloc[0], encoded_df.iloc[1])

print(f"Simple Matching Coefficient: {smc}")
print(f"Cosine Similarity: {cos_sim}")
print(f"Jaccard Similarity: {jac_sim}")
```

Output

```
Cosine Similarity: 0.6200078998918396
Jaccard Similarity: 0.6
Simple Matching Coefficient: 0.6842105263157895
```

Solution

2. Calculating Correlation between ‘Commute Distance’ and ‘Yearly Income’:

```
corr = np.corrcoef(df['NumberCarsOwned'], df['YearlyIncome'])[0, 1]
print(f"Correlation (NumberCarsOwned vs YearlyIncome): {corr}")
```

Output

```
Simple Matching Coefficient: 0.6842105263157895
Correlation (NumberCarsOwned vs YearlyIncome): 0.47730015236316964
```

Assignment 3

Question 1

K-Fold Cross Validation for Multiple Linear Regression (Least Square Error Fit)
Download the dataset regarding USA House Price Prediction from the following link: https://drive.google.com/file/d/1O_NwpJT-8xGfU_-3llUl2sgPu0xllOrX/view?usp=sharing .

Load the dataset and Implement 5- fold cross validation for multiple linear regression (using least square error fit).

(a) Divide the dataset into input features (all columns except price) and output variable (price)
(b) Scale the values of input features
(c) Divide input and output features into five folds.
(d) Run five iterations, in each iteration consider one-fold as test set and remaining four sets as training set. Find the beta (β) matrix, predicted values, and R2_score for each iteration using least square error fit.
(e) Use the best value of (β) matrix (for which R2_score is maximum), to train the regressor for 70% of data and test the performance for remaining 30% data.

Solution

```
from sklearn.model_selection import KFold
from sklearn.preprocessing import StandardScaler
import pandas as pd
import numpy as np

df = pd.read_csv('../assignment_03/USA_Housing.csv')

X = df.drop('Price', axis=1)
y = df['Price']

# print(X.head())
# print(y.head())

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# print(X_scaled[:5])

folds = KFold(n_splits=5, shuffle=True, random_state=42)
# print(folds)

best_r2 = -np.inf

for i, (train_index, test_index) in enumerate(folds.split(X_scaled)):
    X_train, X_test = X_scaled[train_index], X_scaled[test_index]
    y_train, y_test = y[train_index], y[test_index]

    X_train_b = np.c_[np.ones((X_train.shape[0], 1)), X_train]
    X_test_b = np.c_[np.ones((X_test.shape[0], 1)), X_test]

    beta = np.linalg.inv(X_train_b.T.dot(X_train_b)).dot(X_train_b.T).dot(y_train)
    y_pred = X_test_b.dot(beta)
```

```
    ss_total = np.sum((y_test - np.mean(y_test)) ** 2)
    ss_residual = np.sum((y_test - y_pred) ** 2)
    r2_score = 1 - (ss_residual / ss_total)

    if r2_score > best_r2:
        best_r2 = r2_score
        best_beta = beta

    print(f'Iteration {i+1}:')
    print(f'Beta coefficients: {beta}')
    print(f'R2 Score: {r2_score}\n')

print(f'Best R2 Score: {best_r2}')
print(f'Best Beta coefficients: {best_beta}')

split_index = int(0.7 * len(X_scaled))
X_train_final, X_test_final = X_scaled[:split_index], X_scaled[split_index:]
y_train_final, y_test_final = y[:split_index], y[split_index:]

X_train_final_b = np.c_[np.ones((X_train_final.shape[0], 1)), X_train_final]
y_pred_final = np.c_[np.ones((X_test_final.shape[0], 1)), X_test_final].dot(best_beta)

ss_total_final = np.sum((y_test_final - np.mean(y_test_final)) ** 2)
ss_residual_final = np.sum((y_test_final - y_pred_final) ** 2)

r2_score_final = 1 - (ss_residual_final / ss_total_final)
print(f'Final R2 Score on 30% test data: {r2_score_final}')
print(f'Predicted values on 30% test data: {y_pred_final[:5]}')

pred_df = pd.DataFrame(y_pred_final, columns=['Predicted_Price'])
print(X_test_final[:5])
print(pred_df.head())
```

Output

```
Iteration 1:
Beta coefficients: [1232002.6748241   230745.94073479  163243.27314515  120309.77397759
    3011.45976111  151552.63069359]
R2 Score: 0.9179971706985147

Iteration 2:
Beta coefficients: [1232037.85755946  229081.97914235  165882.1605634   121536.57475055
    2092.4478622   150874.99274586]
R2 Score: 0.9145677884802818

Iteration 3:
Beta coefficients: [1231951.92563846  230224.50511001  162766.17455493  121022.77324577
    1247.16258975  150234.77720419]
R2 Score: 0.9116116385364478

Iteration 4:
Beta coefficients: [1232751.46486511  229500.10043209  165212.07110924  122839.9376815
    3063.71699324  150917.88484984]
R2 Score: 0.9193091764960816

Iteration 5:
Beta coefficients: [1.23161736e+06 2.30225051e+05 1.63956839e+05 1.21115120e+05
 7.83467170e+02 1.50662447e+05]
R2 Score: 0.9243869413350316
```

```
Best R2 Score: 0.9243869413350316
Best Beta coefficients: [1.23161736e+06 2.30225051e+05 1.63956839e+05 1.21115120e+05
 7.83467170e+02 1.50662447e+05]
Final R2 Score on 30% test data: 0.9177939588121503
Predicted values on 30% test data: [ 855308.4363391  1279349.31095409 2016682.32498121
1713732.17964167
 1416530.92861757]
[[ 0.40988855 -0.98993223 -1.13805992 -1.33817785 -1.12492995]
 [-0.40282512  2.07744903 -1.29838425 -1.29765967 -0.27789909]
 [ 2.10782492  0.40571791  1.81363115 -0.5116069   0.09301127]
 [ 0.58003921  0.91164992  1.01182501  2.02483141  0.49760458]
 [-0.07629476  0.59473728 -0.01259163 -1.46783604  0.7144602 ]]
   Predicted_Price
0     8.553084e+05
1     1.279349e+06
2     2.016682e+06
3     1.713732e+06
4     1.416531e+06
```

QUESTION 2

Concept of Validation set for Multiple Linear Regression (Gradient Descent Optimization)

Consider the same dataset of Q1, rather than dividing the dataset into five folds, divide the dataset into training set (56%), validation set (14%), and test set (30%). Consider four different values of learning rate i.e. {0.001,0.01,0.1,1}.

Compute the values of regression coefficients for each value of learning rate after 1000 iterations. For each set of regression coefficients, compute R2_score for validation and test set and find the best value of regression coefficients.

SOLUTION

```python
from sklearn.preprocessing import StandardScaler
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

df = pd.read_csv("../assignment_03/USA_Housing.csv")
print(df.head())

X = df.drop("Price", axis=1)
y = df["Price"]

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.44, random_state=42
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)

learning_rates = [0.001, 0.01, 0.1, 1]
```

```
best_r2 = -np.inf
best_model = None

for lr in learning_rates:
    model = LinearRegression()
    model.fit(X_train, y_train)

    r2_val = model.score(X_val, y_val)
    r2_test = model.score(X_test, y_test)

    print(f"Learning Rate: {lr}, R2 Validation: {r2_val}, R2 Test: {r2_test}")

    if r2_val > best_r2:
        best_r2 = r2_val
        best_model = model

print(f"Best R2 on Validation Set: {best_r2}")
print(f"Best Model Coefficients: {best_model.coef_}")
```

Output

```
Avg. Area Income  Avg. Area House Age  Avg. Area Number of Rooms  \
0       79545.45857             5.682861                   7.009188
1       79248.64245             6.002900                   6.730821
2       61287.06718             5.865890                   8.512727
3       63345.24005             7.188236                   5.586729
4       59982.19723             5.040555                   7.839388

   Avg. Area Number of Bedrooms  Area Population         Price
0                          4.09      23086.80050  1.059034e+06
1                          3.09      40173.07217  1.505891e+06
2                          5.13      36882.15940  1.058988e+06
3                          3.26      34310.24283  1.260617e+06
4                          4.23      26354.10947  6.309435e+05
Learning Rate: 0.001, R2 Validation: 0.9175163807040676, R2 Test: 0.9134221506851627
Learning Rate: 0.01, R2 Validation: 0.9175163807040676, R2 Test: 0.9134221506851627
Learning Rate: 0.1, R2 Validation: 0.9175163807040676, R2 Test: 0.9134221506851627
Learning Rate: 1, R2 Validation: 0.9175163807040676, R2 Test: 0.9134221506851627
Best R2 on Validation Set: 0.9175163807040676
Best Model Coefficients: [231827.54854547 166006.22902472 120763.07797071   2922.26769971
 152609.02782229]
```

Solution

```
from sklearn.metrics import r2_score
from sklearn.impute import SimpleImputer
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import pandas as pd
#question("2")[
  Concept of Validation set for Multiple Linear Regression (Gradient Descent
Optimization)

  Consider the same dataset of Q1, rather than dividing the dataset into five folds,
divide the dataset into training set (56%), validation set (14%), and test set (30%).
Consider four different values of learning rate i.e. {0.001,0.01,0.1,1}.

  Compute the values of regression coefficients for each value of learning rate after
1000 iterations.
```

```
  For each set of regression coefficients, compute R2_score for validation and test set
and find the best value of regression coefficients.]
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv("../assignment_03/data.csv")

column_names = [
    "symboling",
    "normalized_losses",
    "make",
    "fuel_type",
    "aspiration",
    "num_doors",
    "body_style",
    "drive_wheels",
    "engine_location",
    "wheel_base",
    "length",
    "width",
    "height",
    "curb_weight",
    "engine_type",
    "num_cylinders",
    "engine_size",
    "fuel_system",
    "bore",
    "stroke",
    "compression_ratio",
    "horsepower",
    "peak_rpm",
    "city_mpg",
    "highway_mpg",
    "price",
]
df.columns = column_names

df.replace("?", np.nan, inplace=True)

# print(df)

figures = {
    "zero": 0,
    "one": 1,
    "two": 2,
    "three": 3,
    "four": 4,
    "five": 5,
    "six": 6,
    "seven": 7,
    "eight": 8,
    "nine": 9,
}

df["num_doors"] = df["num_doors"].map(figures)
df["num_cylinders"] = df["num_cylinders"].map(figures)

df = pd.get_dummies(df, columns=["body_style", "drive_wheels"], drop_first=True)
```

```
label_encoder = LabelEncoder()

df["make"] = label_encoder.fit_transform(df["make"])
df["aspiration"] = label_encoder.fit_transform(df["aspiration"])
df["engine_location"] = label_encoder.fit_transform(df["engine_location"])
df["fuel_type"] = label_encoder.fit_transform(df["fuel_type"])

df["fuel_system"] = df["fuel_system"].apply(
    lambda x: 1 if "pfi" in str(x).lower() else 0
)
df["engine_type"] = df["engine_type"].apply(
    lambda x: 1 if "ohc" in str(x).lower() else 0
)

# imputation
num_cols = df.select_dtypes(include=[np.number]).columns
imputer = SimpleImputer(strategy="mean")
df[num_cols] = imputer.fit_transform(df[num_cols])

cat_cols = df.select_dtypes(exclude=[np.number]).columns
if len(cat_cols) > 0:
    cat_imputer = SimpleImputer(strategy="most_frequent")
    df[cat_cols] = cat_imputer.fit_transform(df[cat_cols])

X = df.drop("price", axis=1)
y = df["price"].astype(float)


scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X = pd.DataFrame(X_scaled, columns=X.columns)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
mse = np.mean((y_test - y_pred) ** 2)
print(f"Mean Squared Error: {mse}")
r2 = r2_score(y_test, y_pred)
print(f"R2 Score: {r2}")

pca = PCA(n_components=0.95)  # retain 95% variance
X_pca = pca.fit_transform(X)
X_train_pca, X_test_pca, y_train, y_test = train_test_split(
    X_pca, y, test_size=0.3, random_state=42
)
model_pca = LinearRegression()
model_pca.fit(X_train_pca, y_train)
y_pred_pca = model_pca.predict(X_test_pca)
mse_pca = np.mean((y_test - y_pred_pca) ** 2)
print(f"Mean Squared Error after PCA: {mse_pca}")
r2_pca = r2_score(y_test, y_pred_pca)
print(f"R2 Score after PCA: {r2_pca}")
```

OUTPUT

```
Mean Squared Error: 20894691.649386063
R2 Score: 0.7438853445721764
Mean Squared Error after PCA: 20620965.335823666
R2 Score after PCA: 0.7472405183003142
```

Assignment 4

Question 1

Write a Python program to scrape all available books from the website (https://books.toscrape.com/) Books to Scrape – a live site built for practicing scraping (safe, legal, no anti-bot). For each book, extract the following details:

1. Title
2. Price
3. Availability (in stock/out of stock)
4. Star rating (One, Two, Three, Four, Five)

Store the scraped results into a Pandas DataFrame and export them to a CSV file named `books.csv`.

Solution

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

books = []
for i in range(1, 11):
    URL = f"https://books.toscrape.com/catalogue/page-{i}.html"
    response = requests.get(URL)
    soup = BeautifulSoup(response.text, "html.parser")

    for article in soup.find_all("article", class_="product_pod"):
        title = article.h3.a.get("title", "").strip()
        price = article.find("p", class_="price_color")
        price = price.text.strip() if price else ""
        availability = article.find("p", class_="instock availability")
        availability = availability.text.strip() if availability else ""
        rating_tag = article.find("p", class_="star-rating")
        rating = rating_tag["class"][1] if rating_tag and len(rating_tag["class"]) > 1
else ""

        book = {
            "title": title,
            "price": price[1:].strip(),
            "availability": availability,
            "rating": rating
        }
        books.append(book)
    print(f"Page {i} scraped successfully.")
df = pd.DataFrame(books)
df.to_csv("books.csv", index=False)
```

Output

```
Page 1 scraped successfully.
Page 2 scraped successfully.
Page 3 scraped successfully.
Page 4 scraped successfully.
Page 5 scraped successfully.
Page 6 scraped successfully.
Page 7 scraped successfully.
```

```
Page 8 scraped successfully.
Page 9 scraped successfully.
Page 10 scraped successfully.
```

Question 2

Write a Python program to scrape the IMDB Top 250 Movies list (https://www.imdb.com/chart/top/) . For each movie, extract the following details:

1. Rank (1–250)
2. Movie Title
3. Year of Release
4. IMDB Rating

Solution

```python
from selenium import webdriver
from selenium.webdriver.common.by import By
import pandas as pd

driver = webdriver.Firefox()
driver.get("https://www.imdb.com/chart/top/")

# ul.ipc-metadata-list
movies = driver.find_elements(By.CSS_SELECTOR, "ul.ipc-metadata-list li")
rows = []
for movie in movies:
    obj = {}
    title_selector = ".ipc-title__text.ipc-title__text--reduced"
    title = movie.find_element(By.CSS_SELECTOR, title_selector).text

    rank = title.split(".")[0]
    obj["Rank"] = rank

    year_selector=".cli-title-metadata-item"
    year = movie.find_element(By.CSS_SELECTOR, year_selector).text

    rating_selector = ".ipc-rating-star--rating"
    rating = movie.find_element(By.CSS_SELECTOR, rating_selector).text

    obj["Rank"] = rank
    obj["Title"] = title.split(". ")[1]
    obj["Year"] = year
    obj["Rating"] = rating
    rows.append(obj)

data = pd.DataFrame(rows)
print(data)

data.to_csv("imdb_top_movies.csv", index=False)
```

Output

```
Rank                     Title  Year Rating
0      1  The Shawshank Redemption  1994    9.3
1      2             The Godfather  1972    9.2
2      3           The Dark Knight  2008    9.1
3      4     The Godfather Part II  1974    9.0
4      5              12 Angry Men  1957    9.0
```

```
..   ...                       ...   ...    ...
120  121                      1917  2019    8.2
121  122                       L.A  1997    8.2
122  123             Oppenheimer   2023    8.3
123  124                Downfall  2004    8.2
124  125          Bicycle Thieves  1948    8.2

[125 rows x 4 columns]
```

QUESTION 3

Write a Python program to scrape the weather information for top world cities from the given website (https://www.timeanddate.com/weather/) . For each city, extract the following details:

1. City Name
2. Temperature
3. Weather Condition (e.g., Clear, Cloudy, Rainy, etc.)

Store the results in a Pandas DataFrame and export it to a CSV file named `weather.csv`.

SOLUTION

```python
import requests
from bs4 import BeautifulSoup
import pandas as pd

URL = "https://www.timeanddate.com/weather/"

headers = {
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/58.0.3029.110 Safari/537.3"
        }
response = requests.get(URL, headers=headers)
soup = BeautifulSoup(response.content, "html.parser")

td = soup.find_all("td")

rows = []
row = {}

for idx, i in enumerate(td):
    if idx % 4 == 0:
        row = {}
        row["City"] = i.text.strip()
        if i.text.strip().endswith("*"):
            row["City"] = row["City"][:-1].strip()
    elif idx % 4 == 1:
        try:
            row["Local Time"] = i.text.split()[1].strip()
        except:
            row["Local Time"] = None
    elif idx % 4 == 2:
        try:
            img = i.find("img")
            row["Weather"] = img["alt"] if img else None
        except:
            row["Weather"] = None
    elif idx % 4 == 3:
```

```
        row["Temperature"] = i.text.strip()
        rows.append(row)

data = pd.DataFrame(rows)
print(data)

data.to_csv("weather.csv", index=False)

# 0: Accra
# 1: सोम 08.58
# 2:
#     img: Overcast. Mild.
# 3: 24 °C
#
```

Output

```
City Local Time                              Weather Temperature
0           Accra      16.04             Scattered clouds. Warm.       29 °C
1     Kuwait City      19.04                        Clear. Mild.       24 °C
2     Addis Ababa      19.04             Scattered clouds. Mild.       17 °C
3            Kyiv      18.04                        Fog. Chilly.        4 °C
4        Adelaide      02.34                               Cool.       16 °C
..            ...        ...                                ...         ...
135        Zagreb      17.04  Light snow. Mostly cloudy. Chilly.        0 °C
136       Kolkata      21.34                         Haze. Mild.       23 °C
137        Zürich      17.04                       Haze. Chilly.       -1 °C
138  Kuala Lumpur      00.04           Passing clouds. Warm.       28 °C
139                     None                                None

[140 rows x 4 columns]
```

Assignment 5

Question 1

Implement Gaussian Naïve Bayes Classifier on the Iris dataset from sklearn.datasets using
(a) Step-by-step implementation.

Solution

```
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split

iris_df = datasets.load_iris()
X = iris_df.data
y = iris_df.target


def mean(X):
    return np.mean(X, axis=0)


def variance(X):
    return np.var(X, axis=0)


def fit(X, y):
    model = {}
    model["classes"] = np.unique(y)
    model["mean"] = {}
    model["var"] = {}
    model["prior"] = {}

    for c in model["classes"]:
        X_c = X[y == c]
        model["mean"][c] = mean(X_c)
        model["var"][c] = variance(X_c)
        model["prior"][c] = X_c.shape[0] / X.shape[0]

    return model


def gaussian_prob(x, mean, var):
    exponent = np.exp(-((x - mean) ** 2) / (2 * var))
    return (1 / np.sqrt(2 * np.pi * var)) * exponent


def predict(model, X):
    y_pred = []
    for x in X:
        class_probs = {}
        for c in model["classes"]:
            prior = np.log(model["prior"][c])
            likelihood = np.sum(
                np.log(gaussian_prob(x, model["mean"][c], model["var"][c]))
            )
```

```
            class_probs[c] = prior + likelihood
        y_pred.append(max(class_probs, key=class_probs.get))
    return np.array(y_pred)


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = fit(X_train, y_train)
y_pred = predict(model, X_test)

print("Predicted labels:", y_pred)
accuracy = np.mean(y_pred == y_test)
print(f"Accuracy: {accuracy * 100:.2f}%")
```

OUTPUT

```
Predicted labels: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Accuracy: 100.00%
```

QUESTION 2

Implement Gaussian Naïve Bayes Classifier on the Iris dataset from sklearn.datasets using
(b) In-built function

SOLUTION

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.naive_bayes import GaussianNB

data = datasets.load_iris()

X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = GaussianNB()
model.fit(X_train, y_train)
accuracy = model.score(X_test, y_test)

y_pred = model.predict(X_test)
print("Predicted labels:", y_pred)
print(f"Accuracy: {accuracy * 100}%")
```

OUTPUT

```
Predicted labels: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]
Accuracy: 100.0%
```

Question 3

Explore about GridSearchCV toot in scikit-learn. This is a tool that is often used for tuning hyperparameters of machine learning models. Use this tool to find the best value of K for K-NN Classifier using any dataset.

Solution

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

data = load_iris()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)
param_grid = {"n_neighbors": np.arange(1, 10)}
print(param_grid)
knn = KNeighborsClassifier()

grid_search = GridSearchCV(knn, param_grid, cv=5)
grid_search.fit(X_train, y_train)
best_k = grid_search.best_params_["n_neighbors"]
print(f"Best value of K: {best_k}")

best_knn = KNeighborsClassifier(n_neighbors=best_k)
best_knn.fit(X_train, y_train)
accuracy = best_knn.score(X_test, y_test)
print(f"Accuracy with best K: {accuracy * 100}%")
```

Output

```
{'n_neighbors': array([1, 2, 3, 4, 5, 6, 7, 8, 9])}
Best value of K: 3
Accuracy with best K: 100.0%
```

Assignment 6

Question 1

Generate a dataset with atleast seven highly correlated columns and a target variable.

Implement Ridge Regression using Gradient Descent Optimization. Take different values of learning rate (such as `0.0001,0.001,0.01,0.1,1,10`) and regularization parameter (10-15,10-10,10-5,10- 3,0,1,10,20). Choose the best parameters for which ridge regression cost function is minimum and R2_score is maximum.

Solution

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("../assignment_07/data.csv")
print(df.head())


def ridge_regression_gradient_descent(X, y, learning_rate, reg_param, num_iterations):
    m, n = X.shape
    theta = np.zeros(n)
    cost_history = []

    for _ in range(num_iterations):
        predictions = X.dot(theta)

        errors = predictions - y

        theta_for_reg = theta.copy()
        theta_for_reg[0] = 0

        gradient = (1 / m) * (X.T.dot(errors)) + (reg_param / m) * theta_for_reg

        theta -= learning_rate * gradient

        cost = (1 / (2 * m)) * np.sum(errors**2) + (reg_param / (2 * m)) * np.sum(
            theta_for_reg**2
        )
        cost_history.append(cost)

    return theta, cost_history


X = df.drop("Target_Variable", axis=1).values
y = df["Target_Variable"].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)

X_train = np.c_[np.ones(X_train.shape[0]), X_train]
X_test = np.c_[np.ones(X_test.shape[0]), X_test]


learning_rates = [0.0001, 0.001, 0.01, 0.1, 1, 10]
reg_params = [1e-15, 1e-10, 1e-5, 1e-3, 0, 1, 10, 20]

best_r2 = -np.inf
best_params = None
best_theta = None
lowest_cost = np.inf

for lr in learning_rates:
    for reg in reg_params:
        theta, cost_history = ridge_regression_gradient_descent(
            X_train, y_train, lr, reg, 1000
        )

        if np.isnan(theta).any():
            continue

        y_pred = X_test.dot(theta)
        r2 = r2_score(y_test, y_pred)
        final_cost = cost_history[-1]

        if r2 > best_r2:
            best_r2 = r2
            best_params = (lr, reg)
            best_theta = theta
            lowest_cost = final_cost

print(f"Best Learning Rate: {best_params[0]}")
print(f"Best Regularization Parameter: {best_params[1]}")
print(f"Best R2 Score: {best_r2:.4f}")
print(f"Cost at Minimum: {lowest_cost:.4f}")
print(f"Best Coefficients (Thetas): \n{best_theta}")
```

Output

```
Feature_1  Feature_2  Feature_3  Feature_4  Feature_5  Feature_6  \
0       10.5       21.2      110.4        5.2       52.1        9.5
1       12.1       24.5      126.3        6.0       60.8       11.2
2        9.8       19.7      103.2        4.9       49.2        8.9
3       15.3       30.8      158.1        7.6       76.9       14.1
4       11.2       22.6      117.5        5.5       56.3       10.4

   Feature_7  Target_Variable
0       30.8            150.2
1       35.9            172.5
2       29.1            140.8
3       45.5            218.9
4       33.2            160.4
/tmp/ipykernel_1883/1529155170.py:28: RuntimeWarning: overflow encountered in square
  cost = (1 / (2 * m)) * np.sum(errors**2) + (reg_param / (2 * m)) * np.sum(
/home/himanshu/assignments/.venv/lib/python3.11/site-packages/numpy/_core/fromnumeric.py:86:
RuntimeWarning: overflow encountered in reduce
  return ufunc.reduce(obj, axis, dtype, out, **passkwargs)
/tmp/ipykernel_1883/1529155170.py:29: RuntimeWarning: overflow encountered in square
```

```
  theta_for_reg**2
/tmp/ipykernel_1883/1529155170.py:24: RuntimeWarning: overflow encountered in dot
  gradient = (1 / m) * (X.T.dot(errors)) + (reg_param / m) * theta_for_reg
/tmp/ipykernel_1883/1529155170.py:17: RuntimeWarning: invalid value encountered in dot
  predictions = X.dot(theta)
/tmp/ipykernel_1883/1529155170.py:24: RuntimeWarning: invalid value encountered in dot
  gradient = (1 / m) * (X.T.dot(errors)) + (reg_param / m) * theta_for_reg
/tmp/ipykernel_1883/1529155170.py:28: RuntimeWarning: invalid value encountered in scalar
multiply
  cost = (1 / (2 * m)) * np.sum(errors**2) + (reg_param / (2 * m)) * np.sum(
/tmp/ipykernel_1883/1529155170.py:24: RuntimeWarning: invalid value encountered in multiply
  gradient = (1 / m) * (X.T.dot(errors)) + (reg_param / m) * theta_for_reg
/tmp/ipykernel_1883/1529155170.py:17: RuntimeWarning: overflow encountered in dot
  predictions = X.dot(theta)
Best Learning Rate: 0.1
Best Regularization Parameter: 1e-15
Best R2 Score: 0.9999
Cost at Minimum: 0.1280
Best Coefficients (Thetas):
[173.7          4.57991815   4.55307975   4.45287262   4.75663862
   4.68334736   3.97350161   4.85169357]
```

QUESTION 2

Load the Hitters dataset from the following link

https://drive.google.com/file/d/1qzCKF6JKKMB0p7ul_lLy8tdmRk3vE_bG/view?usp=sharing

(a) Pre-process the data (null values, noise, categorical to numerical encoding)
(b) Separate input and output features and perform scaling
(c) Fit a Linear, Ridge (use regularization parameter as 0.5748), and LASSO (use regularization parameter as 0.5748) regression function on the dataset.
(d) Evaluate the performance of each trained model on test set. Which model performs the best and Why?

SOLUTION

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score, mean_squared_error

df = pd.read_csv("../assignment_07/hitters.csv")

df = df.dropna(subset=["Salary"])
df = pd.get_dummies(df, columns=["League", "Division", "NewLeague"], drop_first=True)

X = df.drop("Salary", axis=1).values
y = df["Salary"].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=69
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```python
X_test_scaled = scaler.transform(X_test)

lin_reg = LinearRegression()
lin_reg.fit(X_train_scaled, y_train)

ridge_reg = Ridge(alpha=0.5748)
ridge_reg.fit(X_train_scaled, y_train)

lasso_reg = Lasso(alpha=0.5748, max_iter=10000)
lasso_reg.fit(X_train_scaled, y_train)

models = {
    "Linear Regression": lin_reg,
    "Ridge Regression": ridge_reg,
    "Lasso Regression": lasso_reg,
}

print(f"{'Model':<20} | {'R2 Score':<10} | {'RMSE':<10}")

best_model_name = ""
best_r2 = -np.inf

for name, model in models.items():
    y_pred = model.predict(X_test_scaled)

    r2 = r2_score(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))

    print(f"{name:<20} | {r2:.4f}     | {rmse:.2f}")

    if r2 > best_r2:
        best_r2 = r2
        best_model_name = name

print(f"The best performing model is: {best_model_name}")
```

OUTPUT

```
Model                | R2 Score   | RMSE
Linear Regression    | 0.3226     | 338.08
Ridge Regression     | 0.3226     | 338.10
Lasso Regression     | 0.3212     | 338.44
The best performing model is: Linear Regression
```

QUESTION 3

Explore Ridge Cross Validation (RidgeCV) and Lasso Cross Validation (LassoCV)

SOLUTION

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV, LassoCV

# load boston is deprecated :(
# df = load_boston(as_frame=True).frame
data_url = "https://raw.githubusercontent.com/selva86/datasets/master/BostonHousing.csv"
```

```python
df = pd.read_csv(data_url)

X = df.drop("medv", axis=1)
y = df["medv"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)


alphas_to_test = [0.01, 0.1, 1.0, 10.0, 20.0, 50.0, 100.0]

ridge_cv = RidgeCV(alphas=alphas_to_test, cv=5)

ridge_cv.fit(X_train_scaled, y_train)

print(f"Best Alpha found by RidgeCV: {ridge_cv.alpha_}")
print(f"RidgeCV Train Score (R2): {ridge_cv.score(X_train_scaled, y_train):.4f}")
print(f"RidgeCV Test Score (R2):  {ridge_cv.score(X_test_scaled, y_test):.4f}")


lasso_cv = LassoCV(n_alphas=100, cv=5, random_state=42)

lasso_cv.fit(X_train_scaled, y_train)

print(f"Best Alpha found by LassoCV: {lasso_cv.alpha_}")
print(f"LassoCV Train Score (R2): {lasso_cv.score(X_train_scaled, y_train):.4f}")
print(f"LassoCV Test Score (R2):  {lasso_cv.score(X_test_scaled, y_test):.4f}")

coefs = pd.Series(lasso_cv.coef_, index=X.columns)
print(f"Number of features ignored by Lasso (Coef = 0): {sum(coefs == 0)}")
print("Features with 0 weight:", list(coefs[coefs == 0].index))
```

Output

```
Best Alpha found by RidgeCV: 1.0
RidgeCV Train Score (R2): 0.7509
RidgeCV Test Score (R2):  0.6685
Best Alpha found by LassoCV: 0.006863892263379668
LassoCV Train Score (R2): 0.7508
LassoCV Test Score (R2):  0.6684
Number of features ignored by Lasso (Coef = 0): 0
Features with 0 weight: []
/home/himanshu/assignments/.venv/lib/python3.11/site-packages/sklearn/linear_model/
_coordinate_descent.py:1622: FutureWarning: 'n_alphas' was deprecated in 1.7 and will be
removed in 1.9. 'alphas' now accepts an integer value which removes the need to pass
'n_alphas'. The default value of 'alphas' will change from None to 100 in 1.9. Pass an
explicit value to 'alphas' and leave 'n_alphas' to its default value to silence this
warning.
  warnings.warn(
```

Assignment 7

Question 1

Classify SMS messages as:
- spam: (1)
- ham: (0)

Data Description

Column	Meaning
label	spam / ham
text	SMS message content

There are 5500 messages

Part A (Data Preprocessing and Exploration)
1. Load the SMS Spam Collection dataset (spam.csv)
2. Convert label: "spam" → 1, "ham" → 0
3. Text preprocessing:
   - Lowercase
   - Remove punctuation
   - Remove stopwords
4. Convert text to numeric feature vectors using TF-IDF vectorizer
5. Train–test split (80/20)
6. Show class distribution

Part B (Weak Learner Baseline)

Train a Decision Stump: `DecisionTreeClassifier(max_depth: 1)`

Report:
- Train accuracy
- Test accuracy
- Confusion matrix
- Comment briefly on why stump performance is weak on high-dimensional text data

Part C (Manual AdaBoost, T = 15)

Implement AdaBoost manually and after each iteration print:
- Iteration number
- Misclassified sample indices
- Weights of misclassified samples
- Alpha value

Then update and normalize weights.

Also produce:
- Plot: iteration vs weighted error
- Plot: iteration vs alpha

Final report:
- Train accuracy
- Test accuracy
- Confusion matrix
- Short interpretation of weight evolution

Part D (Sklearn AdaBoost)

Train:

```
AdaBoostClassifier(
  base_estimator: DecisionTreeClassifier(max_depth: 1),
  n_estimators: 100,
  learning_rate: 0.6
)
```

Solution

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import string
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

pd.options.mode.chained_assignment = None

DATA_URL = "https://raw.githubusercontent.com/justmarkham/pycon-2016-tutorial/master/
data/sms.tsv"

df = pd.read_csv(
    DATA_URL, sep="\t", header=None, names=["label", "message"], encoding="latin-1"
)

df["label"] = df["label"].map({"ham": 0, "spam": 1})

stopwords_df = pd.read_csv(
    "https://raw.githubusercontent.com/stopwords-iso/stopwords-en/master/stopwords-en.
txt",
    header=None,
)
stopwords = set(stopwords_df[0].values)


def preprocess_text(text):
    """Cleans text by lowercasing, removing punctuation, and stopwords."""
    text = text.lower()
    text = text.translate(str.maketrans("", "", string.punctuation))
    text = " ".join([word for word in text.split() if word not in stopwords])
    return text


df["cleaned_message"] = df["message"].apply(preprocess_text)

X_text = df["cleaned_message"]
y = df["label"]

X_train_text, X_test_text, y_train, y_test = train_test_split(
    X_text, y, test_size=0.2, random_state=42, stratify=y
)

vectorizer = TfidfVectorizer()
X_train = vectorizer.fit_transform(X_train_text)
```

```
X_test = vectorizer.transform(X_test_text)


def evaluate_model(model, X_train, y_train, X_test, y_test, name):
    """Evaluates and prints performance metrics."""
    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)

    train_acc = accuracy_score(y_train, y_train_pred)
    test_acc = accuracy_score(y_test, y_test_pred)
    cm = confusion_matrix(y_test, y_test_pred)

    print(f"Train Accuracy: {train_acc:.4f}")
    print(f"Test Accuracy:  {test_acc:.4f}")
    print("\nConfusion Matrix (Test Set):\n", cm)
    return train_acc, test_acc, cm


stump = DecisionTreeClassifier(max_depth=1, random_state=42)
stump.fit(X_train, y_train)

evaluate_model(stump, X_train, y_train, X_test, y_test, "Decision Stump")

y_train_signed = np.where(y_train == 0, -1, 1)
y_test_signed = np.where(y_test == 0, -1, 1)

N = X_train.shape[0]
T = 15
D = np.full(N, 1 / N)
H_final = np.zeros(N)

error_history = []
alpha_history = []
model_history = []

for t in range(T):
    print(f"\n[Iteration {t + 1}]")

    h_t = DecisionTreeClassifier(max_depth=1, random_state=t)
    h_t.fit(X_train, y_train, sample_weight=D)

    y_pred_01 = h_t.predict(X_train)

    y_pred_signed = np.where(y_pred_01 == 0, -1, 1)

    misclassified_indices = np.where(y_pred_signed != y_train_signed)[0]

    epsilon_t = np.sum(D[misclassified_indices])

    if epsilon_t == 0:
        print("Weighted error is 0. Stopping AdaBoost early.")
        break
    if epsilon_t >= 0.5:
        print(f"Weighted error is {epsilon_t:.4f}. Stump is too weak. Stopping.")
        break

    alpha_t = 0.5 * np.log((1 - epsilon_t) / epsilon_t)

    H_final += alpha_t * y_pred_signed
```

```
    print(f"Weighted Error (ε): {epsilon_t:.4f}")
    print(f"Alpha (α): {alpha_t:.4f}")

    print(
        f"Misclassified samples ({len(misclassified_indices)} total):
{misclassified_indices[:5]}..."
    )
    print(
        f"Initial weights of these samples: {D[misclassified_indices[:5]].round(6)}..."
    )

    D *= np.exp(-alpha_t * y_train_signed * y_pred_signed)
    D /= np.sum(D)

    error_history.append(epsilon_t)
    alpha_history.append(alpha_t)
    model_history.append((alpha_t, h_t))  # Store (alpha, stump model) tuple


def manual_ada_predict(X, model_history):
    """Aggregates predictions from all weak learners."""
    N_samples = X.shape[0]
    final_pred_signed = np.zeros(N_samples)

    for alpha, h_t in model_history:
        y_pred_01 = h_t.predict(X)
        y_pred_signed = np.where(y_pred_01 == 0, -1, 1)
        final_pred_signed += alpha * y_pred_signed

    final_pred_01 = np.where(final_pred_signed > 0, 1, 0)
    return final_pred_01


y_manual_pred_train = manual_ada_predict(X_train, model_history)
y_manual_pred_test = manual_ada_predict(X_test, model_history)

manual_train_acc = accuracy_score(y_train, y_manual_pred_train)
manual_test_acc = accuracy_score(y_test, y_manual_pred_test)
manual_cm = confusion_matrix(y_test, y_manual_pred_test)

print(f"Train Accuracy: {manual_train_acc:.4f}")
print(f"Test Accuracy:  {manual_test_acc:.4f}")
print("\nConfusion Matrix (Test Set):\n", manual_cm)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
iterations = np.arange(1, len(error_history) + 1)

ax1.plot(iterations, error_history, marker="o", linestyle="-", color="red")
ax1.set_title("Iteration vs Weighted Error (ε)")
ax1.set_xlabel("Iteration")
ax1.set_ylabel("Weighted Error (ε)")
ax1.grid(True)

ax2.plot(iterations, alpha_history, marker="o", linestyle="-", color="blue")
ax2.set_title("Iteration vs Alpha (α)")
ax2.set_xlabel("Iteration")
ax2.set_ylabel("Alpha (α) / Classifier Confidence")
ax2.grid(True)

plt.tight_layout()
```

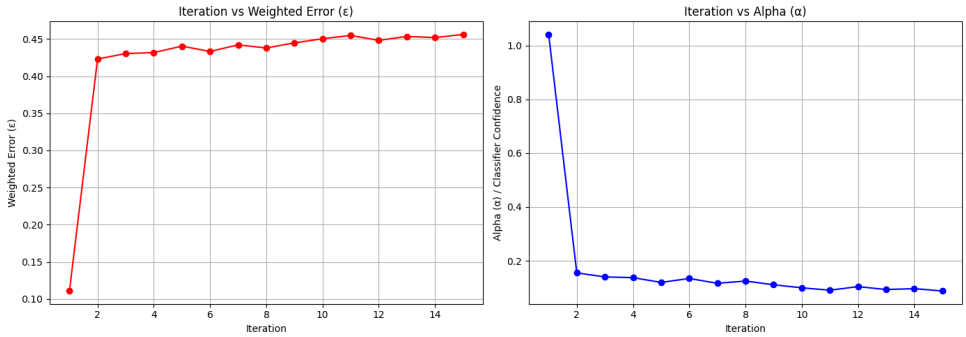
```
plt.show()


ada_clf = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=100,
    learning_rate=0.6,
    random_state=42,
    algorithm="SAMME",
)

ada_clf.fit(X_train, y_train)

sklearn_train_acc, sklearn_test_acc, sklearn_cm = evaluate_model(
    ada_clf, X_train, y_train, X_test, y_test, "Sklearn AdaBoost (100 Est.)"
)

print(f"Manual AdaBoost Test Acc (15 rounds): {manual_test_acc:.4f}")
print(f"Sklearn AdaBoost Test Acc (100 rounds): {sklearn_test_acc:.4f}")
```

Output

```
Train Accuracy: 0.8892
Test Accuracy:  0.8897

Confusion Matrix (Test Set):
 [[965   1]
 [122  27]]

[Iteration 1]
Weighted Error (ε): 0.1108
Alpha (α): 1.0411
Misclassified samples (494 total): [ 9 15 22 31 39]...
Initial weights of these samples: [0.000224 0.000224 0.000224 0.000224 0.000224]...

[Iteration 2]
Weighted Error (ε): 0.4230
Alpha (α): 0.1552
Misclassified samples (509 total): [ 9 15 22 31 39]...
Initial weights of these samples: [0.001012 0.001012 0.001012 0.001012 0.001012]...

[Iteration 3]
Weighted Error (ε): 0.4304
Alpha (α): 0.1401
Misclassified samples (3859 total): [0 1 2 3 4]...
Initial weights of these samples: [0.000109 0.000109 0.000109 0.000109 0.000109]...

[Iteration 4]
Weighted Error (ε): 0.4317
Alpha (α): 0.1374
Misclassified samples (531 total): [ 9 15 22 31 39]...
Initial weights of these samples: [0.00105 0.00105 0.00105 0.00105 0.00105]...

[Iteration 5]
Weighted Error (ε): 0.4404
Alpha (α): 0.1198
Misclassified samples (3859 total): [0 1 2 3 4]...
Initial weights of these samples: [0.000112 0.000112 0.000112 0.000112 0.000112]...

[Iteration 6]
```

```
Weighted Error (ε): 0.4332
Alpha (α): 0.1344
Misclassified samples (550 total): [ 9 15 31 39 49]...
Initial weights of these samples: [0.001087 0.001087 0.001087 0.001087 0.001087]...

[Iteration 7]
Weighted Error (ε): 0.4421
Alpha (α): 0.1164
Misclassified samples (3859 total): [0 1 2 3 4]...
Initial weights of these samples: [0.000112 0.000112 0.000112 0.000112 0.000112]...

[Iteration 8]
Weighted Error (ε): 0.4379
Alpha (α): 0.1248
Misclassified samples (509 total): [ 9 15 22 31 39]...
Initial weights of these samples: [0.001124 0.001124 0.000859 0.001124 0.001124]...

[Iteration 9]
Weighted Error (ε): 0.4448
Alpha (α): 0.1109
Misclassified samples (3859 total): [0 1 2 3 4]...
Initial weights of these samples: [0.000113 0.000113 0.000113 0.000113 0.000113]...

[Iteration 10]
Weighted Error (ε): 0.4503
Alpha (α): 0.0998
Misclassified samples (509 total): [ 9 15 22 31 39]...
Initial weights of these samples: [0.001156 0.001156 0.000883 0.001156 0.001156]...

[Iteration 11]
Weighted Error (ε): 0.4548
Alpha (α): 0.0907
Misclassified samples (3859 total): [0 1 2 3 4]...
Initial weights of these samples: [0.000115 0.000115 0.000115 0.000115 0.000115]...

[Iteration 12]
Weighted Error (ε): 0.4481
Alpha (α): 0.1041
Misclassified samples (531 total): [ 9 15 22 31 39]...
Initial weights of these samples: [0.001177 0.001177 0.000899 0.001177 0.001177]...

[Iteration 13]
Weighted Error (ε): 0.4535
Alpha (α): 0.0933
Misclassified samples (3859 total): [0 1 2 3 4]...
Initial weights of these samples: [0.000115 0.000115 0.000115 0.000115 0.000115]...

[Iteration 14]
Weighted Error (ε): 0.4519
Alpha (α): 0.0966
Misclassified samples (553 total): [ 9 22 49 50 59]...
Initial weights of these samples: [0.001201 0.000918 0.001201 0.00015  0.001201]...

[Iteration 15]
Weighted Error (ε): 0.4562
Alpha (α): 0.0879
Misclassified samples (3859 total): [0 1 2 3 4]...
Initial weights of these samples: [0.000115 0.000115 0.000115 0.000115 0.000115]...
Train Accuracy: 0.8923
Test Accuracy:  0.8933
```

```
Confusion Matrix (Test Set):
 [[965   1]
 [118  31]]
<Figure size 1400x500 with 2 Axes>/home/himanshu/assignments/.venv/lib/python3.11/site-
packages/sklearn/ensemble/_weight_boosting.py:519: FutureWarning: The parameter 'algorithm'
is deprecated in 1.6 and has no effect. It will be removed in version 1.8.
  warnings.warn(
Train Accuracy: 0.9055
Test Accuracy:  0.9022

Confusion Matrix (Test Set):
 [[964   2]
 [107  42]]
Manual AdaBoost Test Acc (15 rounds): 0.8933
Sklearn AdaBoost Test Acc (100 rounds): 0.9022
```



QUESTION 2

Heart Disease Prediction using AdaBoost

Dataset Description You will use the UCI Heart Disease dataset (available in `sklearn.datasets`). This dataset contains patient medical attributes used to predict the presence of heart disease.

Feature	Meaning
Age	Patient age
Sex	Gender (1 = male, 0 = female)
Cp	Chest pain type (0–3)
Trestbps	Resting blood pressure
Chol	Serum cholesterol (mg/dl)
Fbs	Fasting blood sugar >120 mg/dl (1/0)
Restecg	Resting ECG results
Thalach	Maximum heart rate achieved
Exang	Exercise-induced angina (1/0)
Oldpeak	ST depression induced by exercise
Slope	Slope of peak exercise ST segment
Ca	Number of major vessels (0–3)
Thal	Thallium stress test result (0–3)

Target: 1 = Heart disease present 0 = No heart disease

Part A — Baseline Model (Weak Learner)

1. Load and preprocess the dataset (handle categorical features, scaling if needed)
2. Train a single Decision Stump (`max_depth = 1`)
3. Report:

- Training & test accuracy
- Confusion matrix
- Classification report

1. Discuss shortcomings of using only one stump

Part B — Train AdaBoost

1. Train `AdaBoostClassifier` using decision stumps as base learners
2. Use:

- `n_estimators = [5, 10, 25, 50, 100]`
- `learning_rate = [0.1, 0.5, 1.0]`

1. For each combination:

- Train model
- Compute test accuracy

1. Plot:

- **n_estimators** vs **accuracy** for each **learning_rate**

1. Identify the best configuration (highest test accuracy)

Part C — Misclassification Pattern

1. For the best model, collect weak-learner errors and sample weights at each iteration
2. Plot:

- Weak learner error vs iteration
- Sample weight distribution after final boosting stage

1. Explain:

- Which samples receive the highest weights?
- Why does AdaBoost focus more on them?

Part D — Visual Explainability

1. Plot feature importances from AdaBoost
2. Identify the top 5 most important features
3. Explain medically why these features may be strong predictors of heart disease

Solution

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

RANDOM_STATE = 42
```

```
try:
    data = pd.read_csv(
        "https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/
processed.cleveland.data",
        header=None,
        na_values="?",
    )
except Exception:
    print("Could not load data from URL. Using dummy structure.")
    data = pd.DataFrame(np.random.randint(0, 100, size=(303, 14)), columns=range(14))
    data[13] = np.where(data[13] > 40, 1, 0)  # Create dummy target

cols = [
    "age",
    "sex",
    "cp",
    "trestbps",
    "chol",
    "fbs",
    "restecg",
    "thalach",
    "exang",
    "oldpeak",
    "slope",
    "ca",
    "thal",
    "target",
]
data.columns = cols

data["target"] = data["target"].apply(lambda x: 1 if x > 0 else 0)

data = data.dropna()

X = data.drop("target", axis=1)
y = data["target"]

numeric_features = ["age", "trestbps", "chol", "thalach", "oldpeak"]
categorical_features = ["sex", "cp", "fbs", "restecg", "exang", "slope", "ca", "thal"]

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        (
            "cat",
            OneHotEncoder(handle_unknown="ignore", sparse_output=False),
            categorical_features,
        ),
    ],
    remainder="passthrough",
)

X_processed = preprocessor.fit_transform(X)
feature_names = preprocessor.get_feature_names_out()

X_train, X_test, y_train, y_test = train_test_split(
    X_processed, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)
```

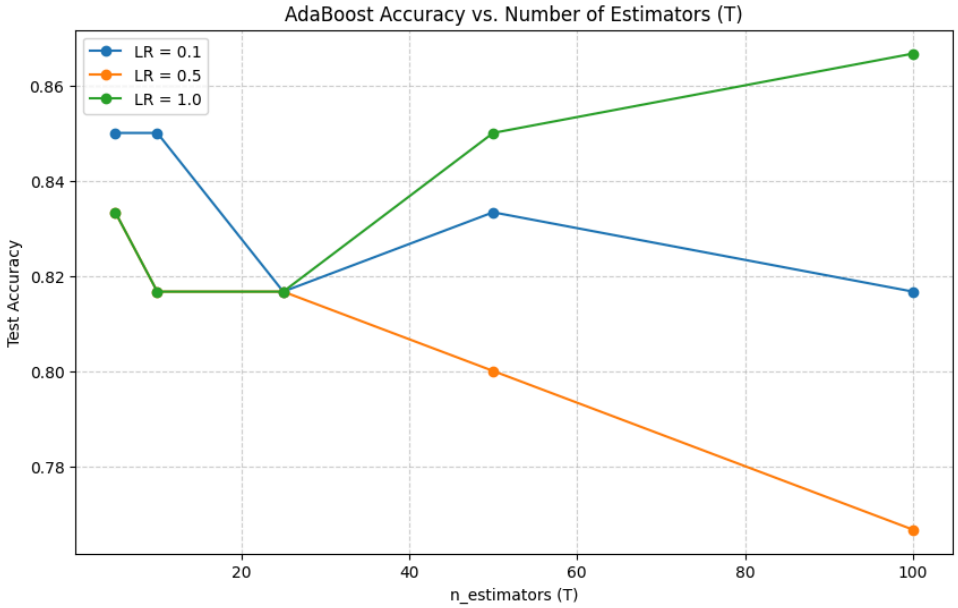
```
print(f"Processed Data Shapes: Train {X_train.shape}, Test {X_test.shape}")


stump = DecisionTreeClassifier(max_depth=1, random_state=RANDOM_STATE)
stump.fit(X_train, y_train)

y_train_pred = stump.predict(X_train)
y_test_pred = stump.predict(X_test)

print(f"Training Accuracy: {accuracy_score(y_train, y_train_pred):.4f}")
print(f"Test Accuracy:     {accuracy_score(y_test, y_test_pred):.4f}")
print("\nConfusion Matrix (Test):\n", confusion_matrix(y_test, y_test_pred))
print("\nClassification Report (Test):\n", classification_report(y_test, y_test_pred))


n_estimators_list = [5, 10, 25, 50, 100]
learning_rate_list = [0.1, 0.5, 1.0]

results = pd.DataFrame(columns=["n_estimators", "learning_rate", "test_accuracy"])
best_acc = 0
best_config = {}

plt.figure(figsize=(10, 6))

for lr in learning_rate_list:
    accuracy_scores = []

    for n_est in n_estimators_list:
        ada_clf = AdaBoostClassifier(
            estimator=DecisionTreeClassifier(max_depth=1, random_state=RANDOM_STATE),
            n_estimators=n_est,
            learning_rate=lr,
            random_state=RANDOM_STATE,
        )

        ada_clf.fit(X_train, y_train)
        test_acc = ada_clf.score(X_test, y_test)

        accuracy_scores.append(test_acc)

        results.loc[len(results)] = [n_est, lr, test_acc]

        if test_acc > best_acc:
            best_acc = test_acc
            best_config = {"n_estimators": n_est, "learning_rate": lr}

    plt.plot(n_estimators_list, accuracy_scores, marker="o", label=f"LR = {lr}")

plt.title("AdaBoost Accuracy vs. Number of Estimators (T)")
plt.xlabel("n_estimators (T)")
plt.ylabel("Test Accuracy")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
plt.savefig("adaboost_accuracy_vs_estimators.png")

print("\nHyperparameter Search Results (Accuracy):")
print(
    results.pivot(
        index="n_estimators", columns="learning_rate", values="test_accuracy"
```

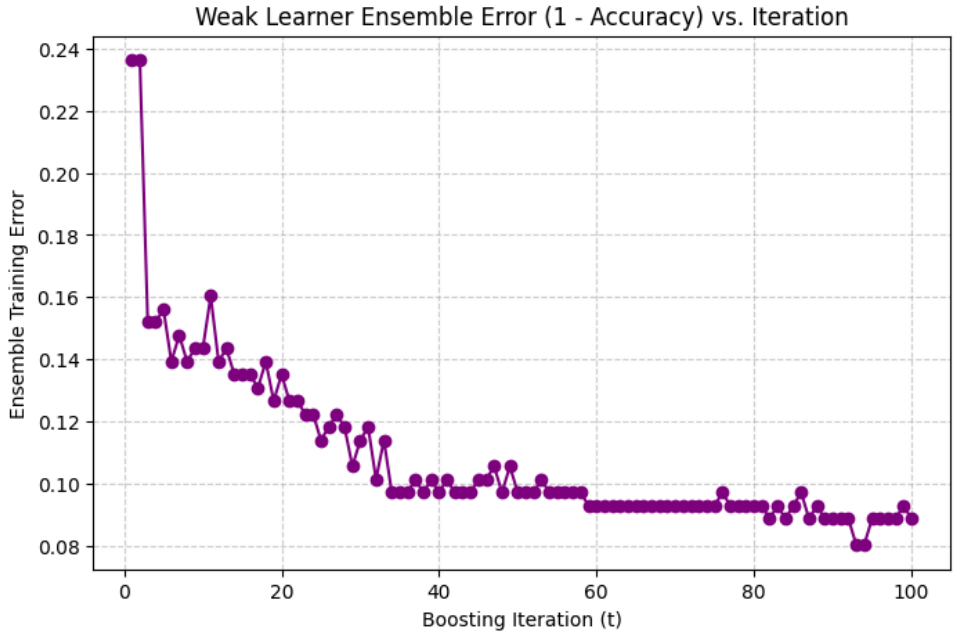
```
    ).round(4)
)
print(
    f"\nIdentified Best Configuration: n_estimators={best_config['n_estimators']},
learning_rate={best_config['learning_rate']} (Accuracy: {best_acc:.4f})"
)
print("-" * 50)

best_model = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1, random_state=RANDOM_STATE),
    n_estimators=best_config["n_estimators"],
    learning_rate=best_config["learning_rate"],
    random_state=RANDOM_STATE,
)
best_model.fit(X_train, y_train)

staged_train_errors = 1 - np.array(list(best_model.staged_score(X_train, y_train)))
staged_iterations = np.arange(1, len(staged_train_errors) + 1)

plt.figure(figsize=(8, 5))
plt.plot(staged_iterations, staged_train_errors, marker="o", color="purple")
plt.title("Weak Learner Ensemble Error (1 - Accuracy) vs. Iteration")
plt.xlabel("Boosting Iteration (t)")
plt.ylabel("Ensemble Training Error")
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
plt.savefig("adaboost_ensemble_error_vs_iteration.png")

final_weights = best_model.estimator_weights_

estimator_weights = best_model.estimator_weights_

plt.figure(figsize=(8, 5))
plt.plot(estimator_weights, marker="o")
plt.title("Estimator Weights (alpha_t) Across Boosting Iterations")
plt.xlabel("Iteration")
plt.ylabel("Alpha (Estimator Weight)")
plt.grid(True, linestyle="--", alpha=0.6)
plt.show()
plt.savefig("adaboost_estimator_weights.png")

importance = best_model.feature_importances_

feature_names_list = list(preprocessor.get_feature_names_out())

feature_importance_df = pd.DataFrame({
    "feature": feature_names_list,
    "importance": importance,
})
feature_importance_df = feature_importance_df.sort_values(
    by="importance", ascending=False
)

plt.figure(figsize=(12, 6))
plt.bar(
    feature_importance_df["feature"][:10],
    feature_importance_df["importance"][:10],
    color="skyblue",
)
plt.xticks(rotation=45, ha="right")
```

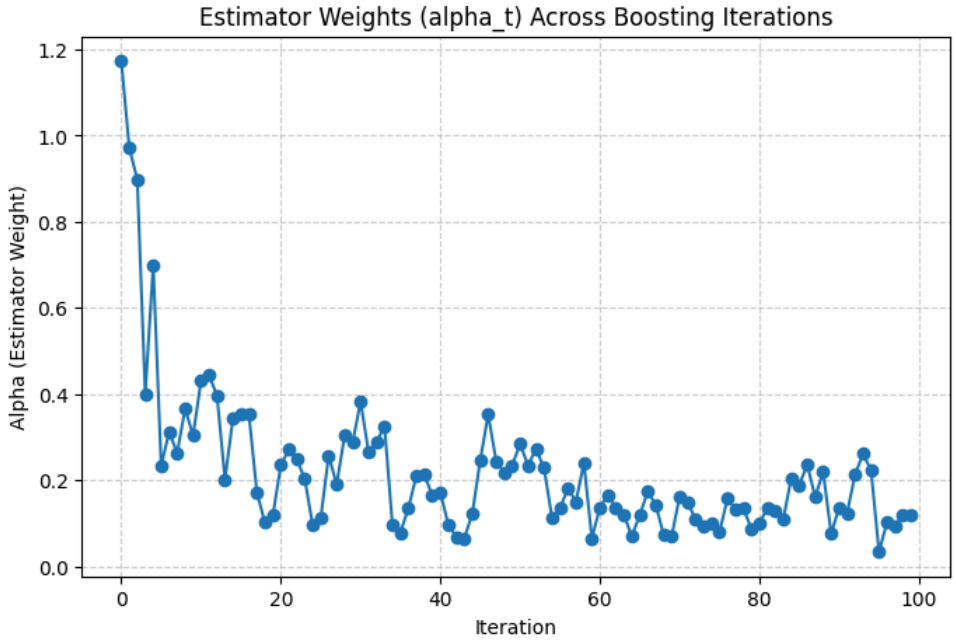
```python
plt.title("Top 10 Feature Importance from AdaBoost Model")
plt.ylabel("Relative Importance")
plt.tight_layout()
plt.show()
plt.savefig("adaboost_feature_importance.png")

top_5_features = feature_importance_df.head(5)
print("\nTop 5 Most Important Features:")
print(top_5_features)
```

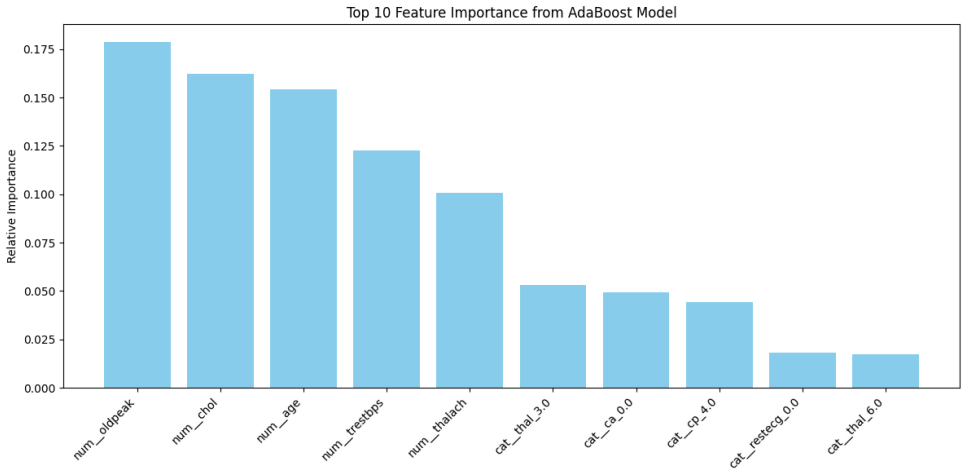
Output

```
Processed Data Shapes: Train (237, 28), Test (60, 28)
Training Accuracy: 0.7637
Test Accuracy:     0.7667

Confusion Matrix (Test):
 [[28  4]
 [10 18]]

Classification Report (Test):
               precision    recall  f1-score   support

           0       0.74      0.88      0.80        32
           1       0.82      0.64      0.72        28

    accuracy                           0.77        60
   macro avg       0.78      0.76      0.76        60
weighted avg       0.77      0.77      0.76        60

<Figure size 1000x600 with 1 Axes>
Hyperparameter Search Results (Accuracy):
learning_rate     0.1     0.5     1.0
n_estimators
5.0            0.8500  0.8333  0.8333
10.0           0.8500  0.8167  0.8167
25.0           0.8167  0.8167  0.8167
50.0           0.8333  0.8000  0.8500
100.0          0.8167  0.7667  0.8667

Identified Best Configuration: n_estimators=100, learning_rate=1.0 (Accuracy: 0.8667)
--------------------------------------------------
<Figure size 640x480 with 0 Axes><Figure size 800x500 with 1 Axes><Figure size 640x480 with
0 Axes><Figure size 800x500 with 1 Axes><Figure size 640x480 with 0 Axes><Figure size
1200x600 with 1 Axes>
Top 5 Most Important Features:
         feature  importance
4   num__oldpeak    0.178848
2      num__chol    0.162175
0       num__age    0.154201
1  num__trestbps    0.122731
3   num__thalach    0.100626
<Figure size 640x480 with 0 Axes>
```


AdaBoost Accuracy vs. Number of Estimators (T)
LR = 0.1
LR = 0.5
LR = 1.0
Test Accuracy
0.86
0.84
0.82
0.80
0.78
20
40
60
80
100
n_estimators (T)


Weak Learner Ensemble Error (1 - Accuracy) vs. Iteration
Ensemble Training Error
0.24
0.22
0.20
0.18
0.16
0.14
0.12
0.10
0.08
0
20
40
60
80
100
Boosting Iteration (t)


Estimator Weights (alpha_t) Across Boosting Iterations
Alpha (Estimator Weight)
Iteration
0.0
0.2
0.4
0.6
0.8
1.0
1.2
0
20
40
60
80
100


Top 10 Feature Importance from AdaBoost Model
Relative Importance
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
num_oldpeak
num_chol
num_age
num_trestbps
num_thalach
cat_thal_3.0
cat_ca_0.0
cat_cp_4.0
cat_restecg_0.0
cat_thal_6.0

Assignment 8

Question 1

The Iris dataset is a classic example for demonstrating classification algorithms. It consists of 150 samples of iris flowers belonging to three species: Setosa, Versicolor, and Virginica, with four input features (sepal and petal length/width). Use SVC from sklearn.svm on the Iris dataset and follow the steps below:

(a) Load the dataset and perform train–test split (80:20).

(b) Train three different SVM models using the following kernels: Linear, Polynomial (degree=3), RBF

(c) Evaluate each model using:
- Accuracy
- Precision
- Recall
- F1-Score

(d) Display the confusion matrix for each kernel.

(e) Identify which kernel performs the best and why

Solution

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
)

iris = datasets.load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
kernels = ["linear", "poly", "rbf"]

results = {}

for kernel in kernels:
    if kernel == "poly":
        model = SVC(kernel=kernel, degree=3)
    else:
        model = SVC(kernel=kernel)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, average="weighted")
    recall = recall_score(y_test, y_pred, average="weighted")
```

```
    f1 = f1_score(y_test, y_pred, average="weighted")
    cm = confusion_matrix(y_test, y_pred)

    results[kernel] = {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1_score": f1,
        "confusion_matrix": cm,
    }

for kernel, metrics in results.items():
    print(f"Kernel: {kernel}")
    print(f"Accuracy: {metrics['accuracy']:.4f}")
    print(f"Precision: {metrics['precision']:.4f}")
    print(f"Recall: {metrics['recall']:.4f}")
    print(f"F1-Score: {metrics['f1_score']:.4f}")
    print("Confusion Matrix:")
    print(metrics["confusion_matrix"])
    print("\n")
```

OUTPUT

```
Kernel: linear
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-Score: 1.0000
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]


Kernel: poly
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-Score: 1.0000
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]


Kernel: rbf
Accuracy: 1.0000
Precision: 1.0000
Recall: 1.0000
F1-Score: 1.0000
Confusion Matrix:
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

QUESTION 2

SVM models are highly sensitive to the scale of input features. When features have different ranges, the algorithm may incorrectly assign higher importance to variables

with larger magnitudes, affecting the placement of the separating hyperplane. Feature scaling ensures that all attributes contribute equally to distance-based computations, which is especially crucial for kernels like RBF or polynomial.

(a) Use the Breast Cancer dataset from sklearn.datasets.load_breast_cancer.

(b) Train an SVM (RBF kernel) model with and without feature scaling (StandardScaler). Compare both results using:
- Training accuracy
- Testing accuracy

(c) Discuss the effect of feature scaling on SVM performance.

Solution

```python
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

data = datasets.load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

svm_no_scaling = SVC(kernel="rbf", random_state=42)
svm_no_scaling.fit(X_train, y_train)
y_train_pred_no_scaling = svm_no_scaling.predict(X_train)
y_test_pred_no_scaling = svm_no_scaling.predict(X_test)
train_accuracy_no_scaling = accuracy_score(y_train, y_train_pred_no_scaling)
test_accuracy_no_scaling = accuracy_score(y_test, y_test_pred_no_scaling)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
svm_with_scaling = SVC(kernel="rbf", random_state=42)
svm_with_scaling.fit(X_train_scaled, y_train)
y_train_pred_with_scaling = svm_with_scaling.predict(X_train_scaled)
y_test_pred_with_scaling = svm_with_scaling.predict(X_test_scaled)
train_accuracy_with_scaling = accuracy_score(y_train, y_train_pred_with_scaling)
test_accuracy_with_scaling = accuracy_score(y_test, y_test_pred_with_scaling)

print("SVM without Feature Scaling:")
print(f"Training Accuracy: {train_accuracy_no_scaling:.4f}")
print(f"Testing Accuracy: {test_accuracy_no_scaling:.4f}")
print("\nSVM with Feature Scaling:")
print(f"Training Accuracy: {train_accuracy_with_scaling:.4f}")
print(f"Testing Accuracy: {test_accuracy_with_scaling:.4f}")
```

Output

```
SVM without Feature Scaling:
Training Accuracy: 0.9143
Testing Accuracy: 0.9474

SVM with Feature Scaling:
```

```
Training Accuracy: 0.9890
Testing Accuracy: 0.9825
```